Report - Recursive Descent Parser

CSCE410101 - Compiler Design
Dr. Ahmed Rafea

By

Basmala Abdelkader 900212406

Mhamad Khalil 900214275

Introduction

In this milestone, we developed a Recursive Descent Parser for Louden's C-language (modified). We used the Lexical Analyzer from the first milestone to tokenize the input. However, some important changes were applied to the lexical analyzer to fit the new requirements and to integrate it smoothly with the parser. These changes, along with the development steps of the parser will be discussed in this report with the test cases and results.

In RDPs, non-terminals in the grammar are all transformed into procedures that call each other matching the Expected input to the Actual one. In this project, we used the C language to develop these procedures, hence the parser, and Flex is still used to generate the C code of the lexical analyzer. The parser's task for now is to only detect syntax errors or marking a successful parsing. That being said, no computation takes place yet.

Syntax and Semantic Rules of C-

The original grammar given in the textbook after applying the changes required by the assignment is as follows:

1. program → **ID** { declaration-list statement-list }
2. declaration-list → declaration-list declaration | declaration
3. declaration → var-declaration (Note: no need for *declaration* anymore)
4. var-declaration → type-specifier **ID** ; | type-specifier **ID** [**NUM**] ;
5. type-specifier → **int** | **float**
6. params → param-list | **void**
7. param-list → param-list , param | param
8. param → type-specifier **ID** | type-specifier **ID [ ]**
9. compound-stmt → **{** statement-list **}**
10. statement-list → statement-list statement | ε
11. statement → assignment-stmt | compound-stmt | selection-stmt | iteration-stmt
12. selection-stmt → **if (** expression **)** statement **if (** expression **)** statement **else** statement
13. iteration-stmt → while **(** expression **)** statement
14. assignment-stmt → var = expression ;
15. var → **ID** | **ID [** expression **]**
16. expression → expression relop additive-expression | additive-expression
17. relop → **<=** | **<** | **>** | **>=** | **==** | **!=**

18. additive-expression →additive-expression addop term | term
19. addop → + | -
20. term → term mulop factor | factor
21. mulop → * | /
22. factor → (expression) | var | **NUM**

The same grammar after removing left-recursion and common prefixes:

1. program → **ID** { declaration-list statement-list }
2. declaration-list → var-declaration declaration-list-tail
3. declaration-list-tail → var-declaration | ε
4. var-declaration → type-specifier **ID** var-declaration-tail
5. var-declaration-tail → **;** | [**NUM**] ;
6. type-specifier → **int** | **float**
7. params → param-list | **void**
8. param-list → param param-list-tail
9. param-list-tail → **,** param param-list-tail | ε
10. param → type-specifier **ID** param-tail
11. param-tail → **[ ]** | ε
12. compound-stmt → **{** statement-list **}**
13. statement-list → statement statement-list | ε
14. statement → assignment-stmt | compound-stmt | selection-stmt | iteration-stmt
15. selection-stmt → **if (** expression **)** statement selection-stmt-tail
16. selection-stmt-tail → **else** statement | ε
17. iteration-stmt → **while (** expression **)** statement
18. assignment-stmt → var = expression ;
19. var → **ID** var-tail
20. var-tail → **[** expression **]** | ε
21. expression → additive-expression expression-tail
22. expression-tail → relop additive-expression expression-tail | ε
23. relop → **<=** | **<** | **>** | **>=** | **==** | **!=**
24. additive-expression →term additive-expression-tail
25. additive-expression-tail → addop term additive-expression-tail | ε
26. addop → + | -
27. term → factor term-tail
28. term-tail → mulop factor term-tail | ε
29. mulop → * | /
30. factor → (expression) | var | **NUM**

Removing left-recursion and common prefixes were important for easier procedure development, especially getting rid of left-recursion that allowed identifying which procedure to call correctly. Every non-terminal ending with "-tail" is added to the grammar for these purposes. For instance, var-declaration-tail was added to handle common prefix "type-specifier **ID**". var-tail was added to remove left-recursion for example.

Changes in the Lexical Analyzer:

The Lexical Analyzer has been modified to take into consideration several things.

- We started with adding the keyword "Program" to the list of existing keywords
- Since we have a file called Tokens.h, which will be explained later, we were able to use the type "Token" in the lexical analyzer, and create a global, shared variable. In the lexical analyzer it's called currentToken. This is the current token matched by the lexical analyzer. So each token matched was now able to return a token name, position, line number, and other details that the parser will use. For example:

```
"int"|"float"|"void"      { currentToken.type = TYPE;
              strcpy(currentToken.lexeme, yytext);
            currentToken.line = line_number;
            currentToken.position = char_position;
            char_position += yyleng;
             printf("Info: Line: %d | Position: %d | 4 \n",
           currentToken.line,
           currentToken.position);
              return TYPE;}
```

One of the tokens defined in Tokens.h is TYPE, and it can be int, void, or float. currentToken is the variable of type Token which is shared between the two files RDP.c and the lexical analyzer. RDP.c can access this shared token.

- We had also changed the mappings; instead of OPERATION, we now have RELOP, ADDOP, MULOP and ASSIGN- an example would be the following:

```
==|!=|<=|>=|>|< { currentToken.type = RELOP;
              strcpy(currentToken.lexeme, yytext);
            currentToken.line = line_number;
            currentToken.position = char_position;
            char_position += yyleng;
printf("Info: Line: %d | Position: %d | 9 \n",
       currentToken.line,
```

```
currentToken.position);
        return RELOP;}
```

- We've also created a specific token for each type of bracket (one for an open curly bracket, one for a closed curly bracket, etc…)

- We've also handled error cases and have a token specific to errors as well.

In terms of Tokens.h:

- This contains definitions for several things:
    - The struct defining the type Token used in both the lexical analyzer and RDP.c
    - The set of tokens defined in an ENUM, we have defined these tokens ourselves based on the rules mentioned and grammar requirements and those of the assignment
    - It contains a get_token_type(TokenType token) function, which is responsible for returning the string representation of a token. So, WHILE would be "WHILE". This was used in the parser rather than the lexical analyzer.
    - The enum includes the following types, along with its definitions
        - (Keywords) PROGRAM, IF, ELSE, WHILE, TYPE, RETURN
        - (Identifiers and Literals) ID and NUM
        - (Symbols and punctuation) OPEN_PAR, CLOSED_PAR, SEMICOLON, COMMA, OPEN_CURL (curly brackets)...
        - (Operators) ADDOP, RELOP, MULOP, ASSIGN
        - (Special Tokens) END_OF_FILE and ERROR

Parser Design

The parser uses a Recursive Descent approach to perform syntax analysis. It verifies that the input adheres to the language's grammatical rules.
Every non-terminal is written as a procedure with the corresponding logic of the production rule.
Some helper functions were introduced as well:

- void match(TokenType expected): checks the type of the current token (lookahead) and compares it to the expected token type. If they match, then the function moves to the next token. Otherwise, a syntax error is raised.
- syntax_error(TokenType expected): prints an error message to the user telling them the expected token type, the one received by the parser, the line and the character position of the error.

In order to let the parser consume the tokens detected by the lexical analyzer, the RDP uses external functions and variables (from the lexical analyzer).

- yylex(): defined and used in the lexical analyzer. It is called by the parser to move between detected tokens. In other words, match uses it to move to the next token. It

returns an int (which refers to the token type internally defined by flex). This integer is used at the end as a flag (int status) to check if the parsing was successful or not.

- currentToken: a variable of type Token (struct) defined and filled in the lexical analyzer. This variable holds all the information needed about a token: type, lexeme, line number, and character position.
- yyin: used by the lexical analyzer to store the input file. The parser passes the input file from its arguments to yyin to be consumed by the scanner.

Assumptions

The parser should work as expected without any problems if these assumptions were met:

1. The C- program must start with the keyword "Program" (case sensitive). In other words, the starting production rule of the grammar is:
    a. program → **ID** { declaration-list statement-list }
2. The parser does not support procedure calling or function creation in the input as the rules supporting these were removed.
3. The only data types supported are **int** and **float.** The use of any other data type will result in having an "UNKNOWN_TOKEN", except for using **void** which will be detected as a "TYPE" however cannot be used with identifiers and left to satisfy some unused grammar rules (ex. params).
4. Same conventions used in the previous milestone still hold:
    a. Letter definition (reg-ex): [a-zA-Z]
    b. Digit definition (reg-ex): [0-9]
    c. ID definition (reg-ex): Letter (Letter | Digit)* (("@"|"$" | "_")? Digit+)?
    d. Num definition (reg-ex): Digit (".")? Digit+((E |e) (+|-)? Digit+)?
    e. The language is case sensitive
    f. Whitespaces (alongside Newline "\n" and Tab "\t") should be ignored and used to separate tokens
5. Based on the definition specified on Canvas, NUM = digit (".")? digit+((E |e) (+|-)? digit+)?, we've assumed that only one digit can come before the decimal point, but it can have as many digits as needed after the decimal point.
6. One input file is expected to be provided, hence the use of "%option noyywrap"

Note: some problems from previous milestones were fixed: (from previous report)

1. All special symbols (reserved symbols) must be separated by a space (or tab or newline) if it comes after a digit, such that 9; would be considered a wrong number, but 9 ; would be correct, where 9 is considered a number and ; is considered a reserved character.
2. A number followed by an Identifier is not handled at this stage of the project and is considered "NUM ID". Handling this case is left to the Parser.

These two points are now handled and work fine.

Steps to Run:

If lex.yy.c was in the directory you can skip the second step.

1. Use a txt file (in this case input.txt) to store the code for testing.
2. Run the command: flex lexicalAnalyzer.l and click enter.
    a. This generates a .c file called “lex.yy.c”
3. DO NOT compile lex.yy.c as it does not contain a main function and cannot be used alone
4. Compile the Parser: gcc RDP.c -o <output-name>
5. Run the executable with the correct argument: ./<output-name> input.txt

Test Cases:

Input:

```
Program myArrayProg {
   int nums[10];
}
```

Output:

```
TOKEN: PROGRAM        | Line: 1 | Pos: 1 | Lexeme: Program
TOKEN: ID             | Line: 1 | Pos: 9 | Lexeme: myArrayProg
TOKEN: OPEN_CURL      | Line: 1 | Pos: 21 | Lexeme: {
TOKEN: TYPE           | Line: 2 | Pos: 5 | Lexeme: int
TOKEN: ID             | Line: 2 | Pos: 9 | Lexeme: nums
TOKEN: OPEN_SQUARE    | Line: 2 | Pos: 13 | Lexeme: [
TOKEN: NUM            | Line: 2 | Pos: 14 | Lexeme: 10
TOKEN: CLOSED_SQUARE  | Line: 2 | Pos: 16 | Lexeme: ]
TOKEN: SEMICOLON      | Line: 2 | Pos: 17 | Lexeme: ;
TOKEN: CLOSED_CURL    | Line: 3 | Pos: 1 | Lexeme: }
State:0
Parsing was successful!
```

Input:

```
Program ifTest {
   int x;
   if (x) {
      x = 15;
   }
}
```

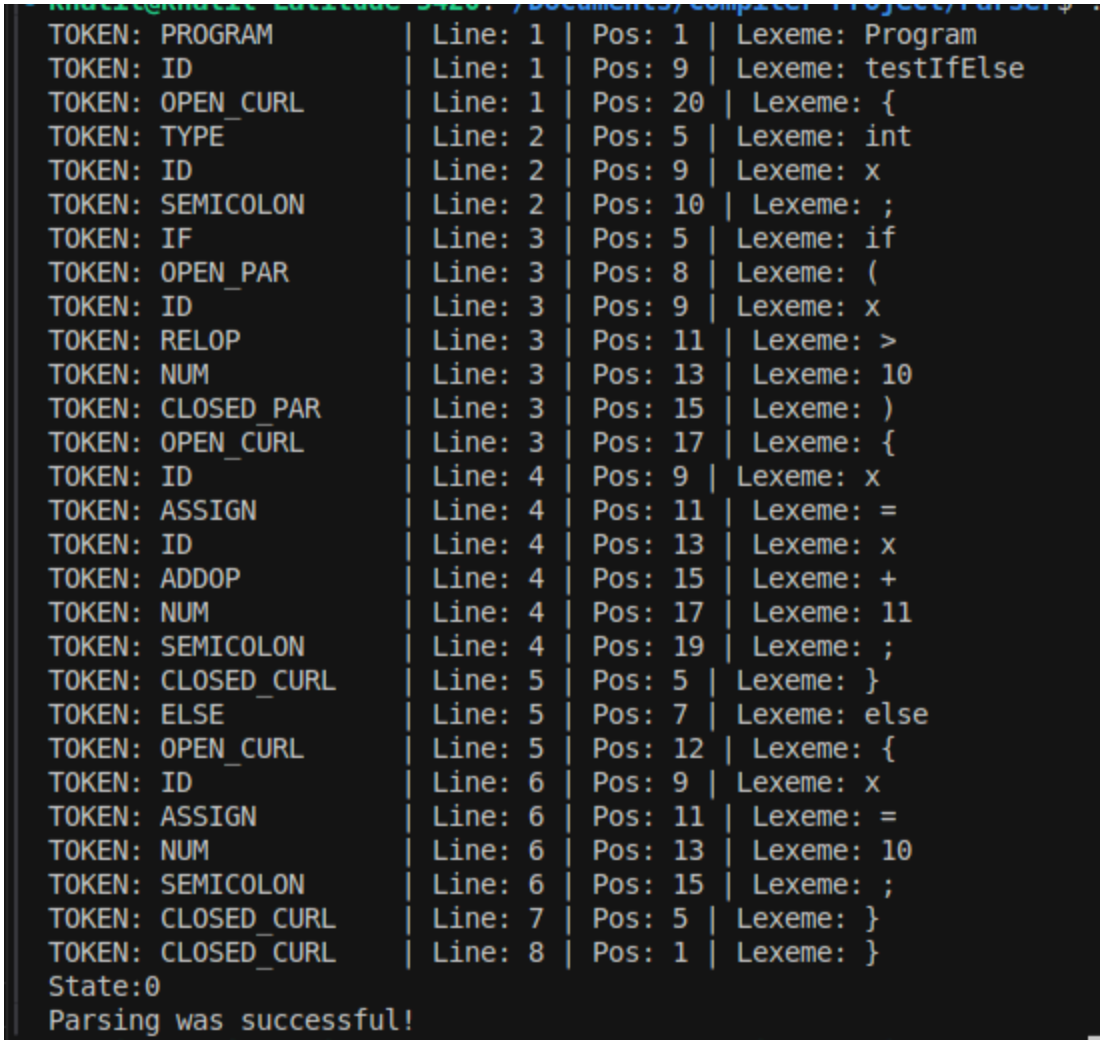
Output:

```
TOKEN: PROGRAM        | Line: 1 | Pos: 1 | Lexeme: Program
TOKEN: ID             | Line: 1 | Pos: 9 | Lexeme: ifTest
TOKEN: OPEN_CURL      | Line: 1 | Pos: 16 | Lexeme: {
TOKEN: TYPE           | Line: 2 | Pos: 5 | Lexeme: int
TOKEN: ID             | Line: 2 | Pos: 9 | Lexeme: x
TOKEN: SEMICOLON      | Line: 2 | Pos: 10 | Lexeme: ;
TOKEN: IF             | Line: 3 | Pos: 5 | Lexeme: if
TOKEN: OPEN_PAR       | Line: 3 | Pos: 8 | Lexeme: (
TOKEN: ID             | Line: 3 | Pos: 9 | Lexeme: x
TOKEN: CLOSED_PAR     | Line: 3 | Pos: 10 | Lexeme: )
TOKEN: OPEN_CURL      | Line: 3 | Pos: 12 | Lexeme: {
TOKEN: ID             | Line: 4 | Pos: 9 | Lexeme: x
TOKEN: ASSIGN         | Line: 4 | Pos: 11 | Lexeme: =
TOKEN: NUM            | Line: 4 | Pos: 13 | Lexeme: 15
TOKEN: SEMICOLON      | Line: 4 | Pos: 15 | Lexeme: ;
TOKEN: CLOSED_CURL    | Line: 5 | Pos: 5 | Lexeme: }
TOKEN: CLOSED_CURL    | Line: 6 | Pos: 1 | Lexeme: }
State:0
Parsing was successful!
```

Input:

```
Program testIfElse {
   int x;
   if (x > 10) {
      x = x + 11;
   } else {
      x = 10;
   }
}
```

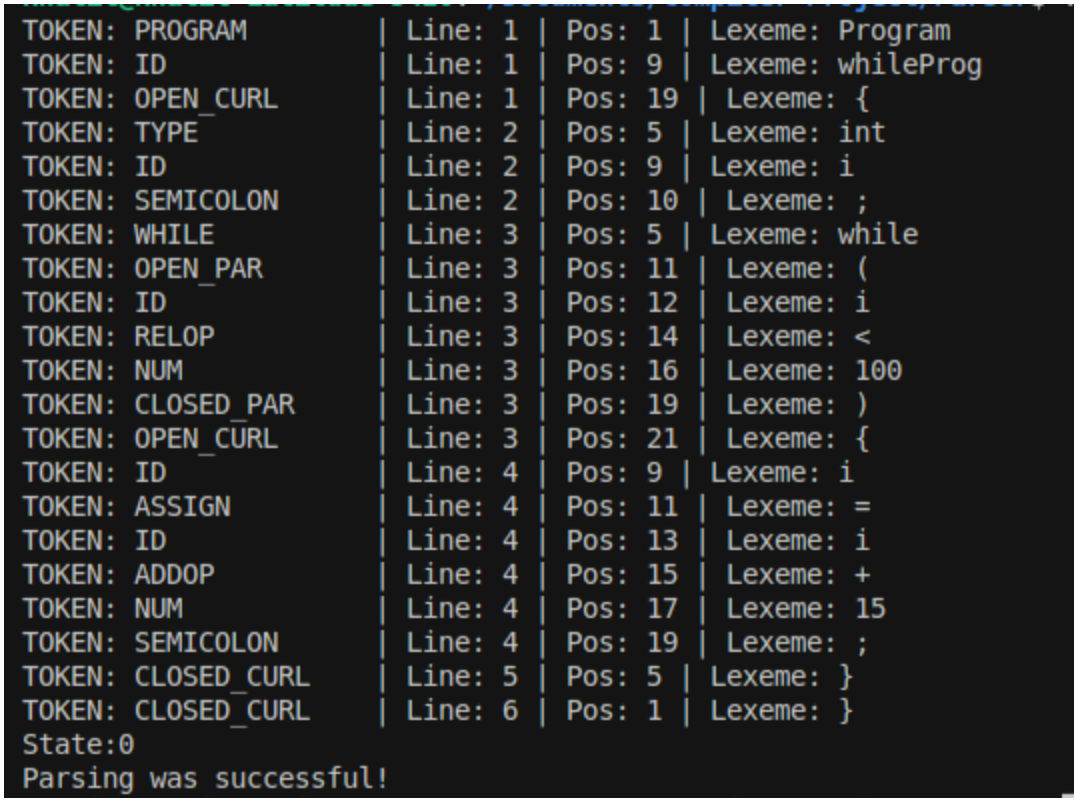
Output:



Input:

```
Program whileProg {
   int i;
   while (i < 100) {
      i = i + 15;
   }
}
```

Output:



Implementation Issues

- We first had trouble understanding exactly what we needed to modify in the lexical analyzer to link it with the parser, and how we could use them both together, since the lexical analyzer used to only print an output, not actually return anything.
- We had trouble also knowing which tokens to map to each other at first and which to keep separate, such as the open and closed bracket types
- The third issue we had was being able to detect a number or identifier if immediately followed by a symbol like brackets, semicolon, etc.
- The fourth and more challenging one was figuring out how to link both files and run them so that they work together to tokenize the code and track it, and one of the solutions to this was the global shared variable, and the second was Tokens.h.
- We also had to add/ remove rules, or modify existing rules to fit the requirements of this assignment.
- Assumptions have been made, and are specified above in the Assumptions section.

All these issues had been resolved successfully.